§ Contents

1 Overview

Single-Company Bus Operator | 24-Table Production Schema

2 Table Inventory — Plain-English Guide

3 Table Assessment & New Table Proposals

4 Benchmarking Against Real Bus-Booking Platforms

5 Prioritized Table Build Order

6 Data Model Summary — Relationships

7 Entity-Relationship Diagram (All 24 Tables)

8 Process Flow Diagrams (Booking, Payment, Cancellation)

9 Full Column-Level Table Definitions

10 Assumptions & Constraints

1. Overview

This document defines the database design for a single-company bus booking and cancellation application. The company owns and operates its own fleet — there is no multi-operator marketplace logic, which keeps the schema lean and focused purely on the booking lifecycle: search, seat selection, payment, and cancellation.

The design covers **24 core tables** grouped into six functional areas: Location & Route Management, Fleet Management, Scheduling, User Management, Booking & Payments, and Growth & Operations (added after benchmarking against RedBus, AbhiBus, and Yatra-style platforms — see Section 4). Optional features like multi-operator support remain clearly flagged rather than forced into the single-company scope.

Functional Area	Count	Tables
Location & Route	4	Cities, Terminals, Routes, Route_stops
Fleet Management	4	Bus_types, Buses, Seat_layout, Bus_amenities
Scheduling	3	Drivers, Trips, Trip_seats
User Management	2	Users, Passengers
Booking & Payments	5	Bookings, Booking_passengers, Payments, Cancellations, Refunds
Growth & Operations	6	Coupons, Booking_coupons, Notifications, Reviews_ratings, Admin_users, Operators

2. Table Inventory — Plain-English Guide

This table explains each entity in everyday language, without SQL syntax, so it can be shared with non-technical stakeholders.

Entity	What It Stores (Plain Words)	Connects To
City	A serviceable town/city	Terminals, Routes
Terminal	A physical bus stand in a city	Cities, Routes
Bus Type	AC / Non-AC / Sleeper category	Buses
Bus	A physical bus in the company fleet	Bus Type, Seat Layout, Trips
Seat Layout	The seat map of a bus	Buses, Trip Seats
Route	A path from one city to another	Cities, Route Stops, Trips
Route Stop	A stop along a route	Routes, Cities
Trip	A scheduled bus run on a date/time	Route, Bus, Bookings
Trip Seat	Live seat status for a specific trip	Trip, Seat Layout
User	A customer account	Passengers, Bookings
Passenger	A person traveling (may not be the account holder)	Users, Bookings
Booking	A confirmed reservation	User, Trip, Payment, Cancellation
Booking Passenger	Which passenger sits in which seat, for which booking	Booking, Passenger, Trip Seat
Payment	Money paid for a booking	Booking
Cancellation	A cancelled booking and its refund info	Booking
Driver	A driver/conductor assigned to a trip	Trips
Refund	The actual money-back transaction for a cancellation	Cancellation, Payment
Coupon	A discount code and its rules	Booking Coupon
Booking Coupon	Which coupon was used on which booking	Booking, Coupon
Notification	A sent SMS/email/push message log	User, Booking
Review/Rating	A passenger's post-trip feedback	Booking, User, Trip
Admin User	A staff account for backend operations	(standalone)
Operator	A bus company — optional, for multi-operator expansion	(standalone)

3. Table Assessment & New Table Proposals

Confirmed Present

All 15 originally listed tables (Cities, Terminals, Routes, Route_stops, Buses, Seat_layout, Bus_amenities, Trips, Trip_seats, Users, Passengers, Bookings, Booking_passengers, Payments, Cancellations) are present and cover the full booking lifecycle end-to-end.

Added — Core Gap

> **Bus_types** — Classifies each bus (AC / Non-AC / Sleeper / Seater) as a controlled reference list instead of free text, preventing inconsistent entries like "AC" vs "A/C".
> Keys: `bus_type_id` (PK) → referenced by `Buses.bus_type_id` (FK). Relationship: one-to-many.

Added — Benchmarked Against Real Platforms

The tables below were not in the original 15 and were not in the first proposal either — they are added now after checking how real bus-booking platforms structure their systems (see Section 4 for the full rationale and sourcing).

- **Drivers** — Assigns a driver/conductor to each Trip for accountability and emergency contact. Keys: `driver_id` (PK) → referenced by `Trips.driver_id` (FK). Relationship: one-to-many.
- **Refunds** — Tracks the actual refund transaction separately from the cancellation decision, since a refund can go through multiple processing states after a Cancellation is approved. Keys: `refund_id` (PK) → `cancellation_id`, `payment_id` (FK).
- **Coupons and Booking_coupons** — Discount code management, standard on every major booking platform. Keys: `coupon_id` (PK); Booking_coupons resolves the many-to-many between Bookings and Coupons.
- **Notifications** — Logs SMS/email/push confirmations and reminders sent to users. Keys: `notification_id` (PK) → `user_id`, `booking_id` (FK).
- **Reviews_ratings** — Post-trip feedback, a core feature on RedBus-style platforms. Keys: `review_id` (PK) → `booking_id`, `user_id`, `trip_id` (FK).
- **Admin_users** — Backend staff accounts for operations and support, separate from customer Users. Keys: `admin_id` (PK), independent table.
- **Operators** — Included for completeness since most real-world platforms are multi-operator marketplaces. Marked optional — not required while this remains a single-company app; add only if the business later onboards other bus companies.

4. Benchmarking Against Real Bus-Booking Platforms

To check for gaps, this design was compared against publicly documented case studies and open-source implementations of bus reservation systems modeled on RedBus-style platforms.

- A widely-referenced RedBus system design breakdown describes the platform as composed of several cooperating services — user-facing search, booking, payment, live bus tracking, and a dedicated review service — confirming that review/rating functionality is treated as a first-class, standalone concern rather than bolted onto Bookings directly.
- A RedBus case study describes its post-trip feedback flow: a user opens a past booking and submits a 1–5 rating with a written comment, which is stored as its own record and aggregated into per-service scores — matching the standalone Reviews_ratings design used here.
- An open-source bus reservation schema confirms a dedicated Driver entity referenced from the trip/schedule table, supporting the addition of Drivers as a foreign key on Trips rather than a free-text field.
- A separate open-source bus ticketing system's schema shows Payment records linked to both a Booking and an Operator, with a distinct BusOperator entity referenced from the Bus table — which is why Operators is included here for completeness, even though it stays optional for a single-company build.

> **Net result:** the original 16-table design already covered the core booking lifecycle correctly. The 8 additions above are the pieces that separate an MVP from a platform that matches what RedBus, AbhiBus, and Yatra actually run in production — discounting, notifications, reviews, refunds as their own lifecycle, driver accountability, staff access, and (optionally) multi-operator support.

Still Optional — Not Added (Flagged for v3, Not v2)

- **Wallet / Wallet_transactions** — store-credit balance, used by RedBus for faster refunds. Adds meaningful complexity (balance consistency, transaction ledger) — worth adding only once refund volume justifies it.
- **Support_tickets** — customer support/helpdesk ticketing, a separate concern from the booking flow itself.
- **Bus_documents** — insurance, permit, and fitness certificate tracking for compliance. Useful for fleet management, not for the booking-cancellation flow this schema is scoped to.

5. Prioritized Table Build Order

Tables are ordered so that every foreign key reference already exists by the time it's needed — this is the safe order to run `CREATE TABLE` statements in. 24 tables total, including the 8 benchmarked additions from Section 4.

Step 1 — Location & Route Foundation

1. **Cities** — root of the hierarchy; every location-based table depends on it.
2. **Terminals** — depends on Cities; defines physical boarding locations.

Step 2 — Fleet Setup

3. **Bus_types** — independent master table; needed before Buses can reference it.
4. **Buses** — depends on Bus_types; defines the fleet.
5. **Seat_layout** — depends on Buses; defines each bus's seat map.
6. **Bus_amenities** — depends on Buses; enriches bus data, non-blocking for booking flow.

Step 3 — Route Definition

7. **Routes** — depends on Cities; defines source-destination paths.
8. **Route_stops** — depends on Routes and Cities/Terminals; defines the stop sequence.

Step 4 — Scheduling Engine

9. **Drivers** — independent master table; needed before Trips can optionally reference it.
10. **Trips** — depends on Routes, Buses, and Drivers; the sellable/bookable unit — schema's centerpiece.
11. **Trip_seats** — depends on Trips and Seat_layout; tracks live seat availability.

Step 5 — Users

12. **Users** — independent; needed before any booking can be attributed to someone.
13. **Passengers** — depends on Users, since a user books for a group.

Step 6 — Transactions

14. **Bookings** — depends on Users and Trips; the core transaction record.
15. **Booking_passengers** — depends on Bookings, Passengers, and Trip_seats; links who sits where.
16. **Payments** — depends on Bookings; records money movement.
17. **Cancellations** — depends on Bookings; closes the lifecycle loop.
18. **Refunds** — depends on Cancellations and Payments; tracks the settlement transaction.

Step 7 — Growth & Operations (Benchmarked Additions)

19. **Coupons** — independent master table; discount code rules.
20. **Booking_coupons** — depends on Bookings and Coupons; links a redemption to a booking.
21. **Notifications** — depends on Users and Bookings; logs sent messages.
22. **Reviews_ratings** — depends on Bookings, Users, and Trips; post-trip feedback.
23. **Admin_users** — independent; backend staff accounts.
24. **Operators** — independent; optional, only needed for multi-operator expansion.

6. Data Model Summary

Key-entity map showing primary keys (PK), foreign keys (FK), and relationship cardinality.

Relationship	Type	Keys Involved
Cities → Terminals	1:M	`city_id` (PK) → `Terminals.city_id` (FK)
Bus_types → Buses	1:M	`bus_type_id` (PK) → `Buses.bus_type_id` (FK)
Buses → Seat_layout	1:M	`bus_id` (PK) → `Seat_layout.bus_id` (FK)
Buses → Bus_amenities	1:M	`bus_id` (PK) → `Bus_amenities.bus_id` (FK)
Cities → Routes	1:M each	`city_id` (PK) → `Routes.source_city_id` / `destination_city_id` (FK)
Terminals → Routes	1:M each	`terminal_id` (PK) → `Routes.source_terminal_id` / `destination_terminal_id` (FK)
Routes → Route_stops	1:M	`route_id` (PK) → `Route_stops.route_id` (FK)
Routes + Buses → Trips	M:N resolved	`route_id` + `bus_id` (FK) → Trips (junction-style scheduling table)
Drivers → Trips	1:M (optional)	`driver_id` (PK) → `Trips.driver_id` (FK)
Trips + Seat_layout → Trip_seats	1:M	`trip_id` + `seat_id` (FK) → Trip_seats
Users → Passengers	1:M	`user_id` (PK) → `Passengers.user_id` (FK)
Users + Trips → Bookings	1:M each	`user_id` + `trip_id` (FK) → Bookings
Bookings ↔ Passengers ↔ Trip_seats	M:N resolved	Booking_passengers (`booking_id` , `passenger_id` , `trip_seat_id`)
Bookings → Payments	1:1 (typical)	`booking_id` (PK) → `Payments.booking_id` (FK)
Bookings → Cancellations	1:1 (conditional)	`booking_id` (PK) → `Cancellations.booking_id` (FK)
Cancellations + Payments → Refunds	1:1 each	`cancellation_id` + `payment_id` (FK) → Refunds
Bookings ↔ Coupons	M:N resolved	Booking_coupons (`booking_id` , `coupon_id`)
Users + Bookings → Notifications	1:M each	`user_id` + `booking_id` (FK) → Notifications
Bookings + Users + Trips → Reviews_ratings	1:1 / 1:M	`booking_id` / `user_id` / `trip_id` (FK) → Reviews_ratings

Note on Trips: Trips is the schema's centerpiece — it resolves the many-to-many relationship between Routes and Buses into concrete, bookable instances (a specific bus, on a specific route, at a specific date/time).

7. Entity-Relationship Diagram — All 24 Tables

The diagram below has been redrawn from scratch for clarity and to correct crossing/ambiguous connectors in the original export. Tables are grouped by functional area (matching Section 1), and every connector uses standard crow's-foot notation: a single bar means "one," a crow's foot means "many." All relationships are verified against the column-level foreign keys in Section 9.

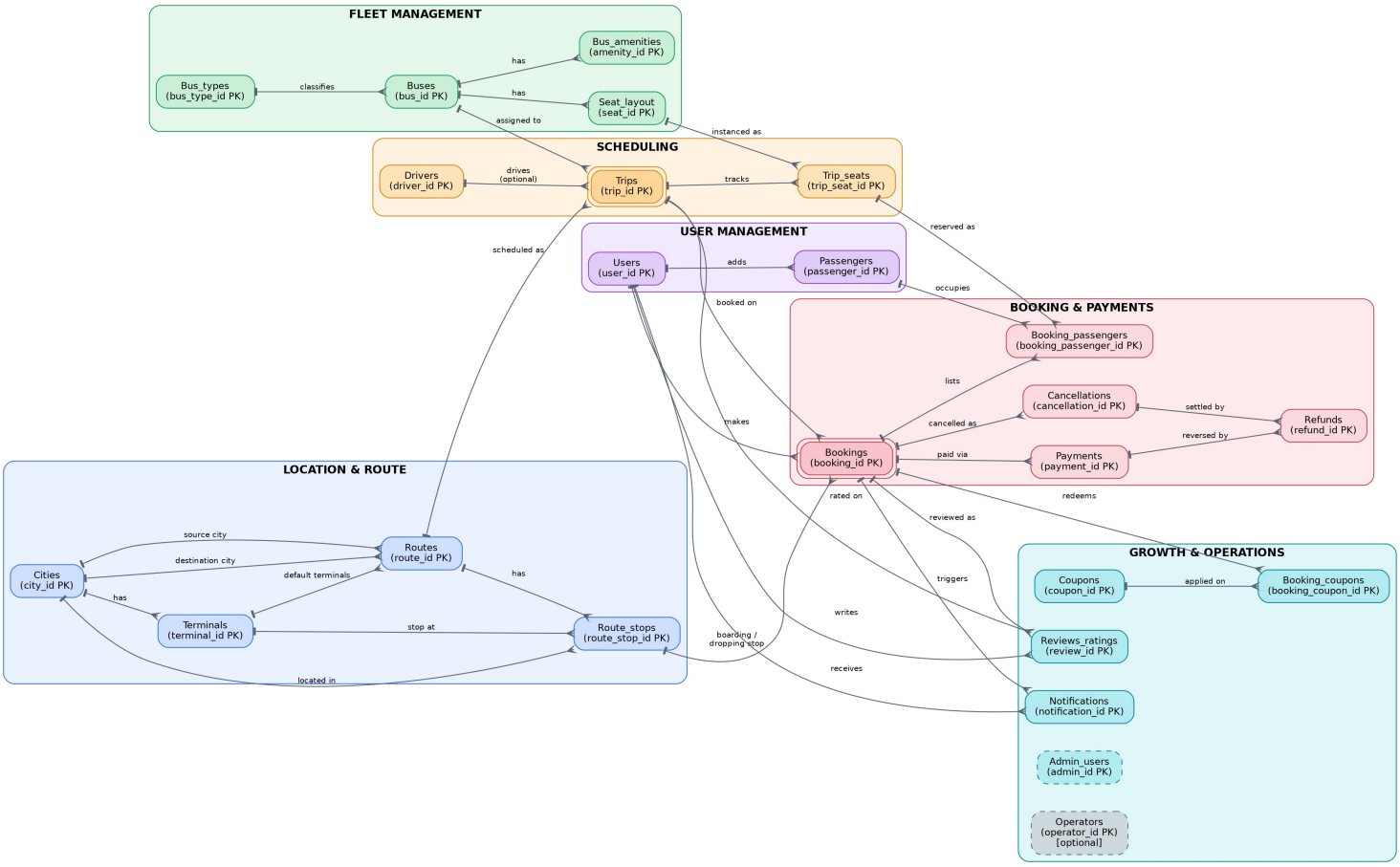


Figure 1 — Corrected entity-relationship diagram, all 24 tables, grouped by functional area. Double-bordered nodes (Trips, Bookings) mark the two central "hub" entities the rest of the schema radiates from.

8. Process Flow Diagrams

Three flows cover the full booking lifecycle end-to-end. Flow A (Book a Trip) was incomplete/missing its steps in the original source document as a rendering artifact — it has been reconstructed below from the schema's own seat-holding and booking logic described in Section 10 (Assumptions & Constraints).

Flow A — Book a Trip

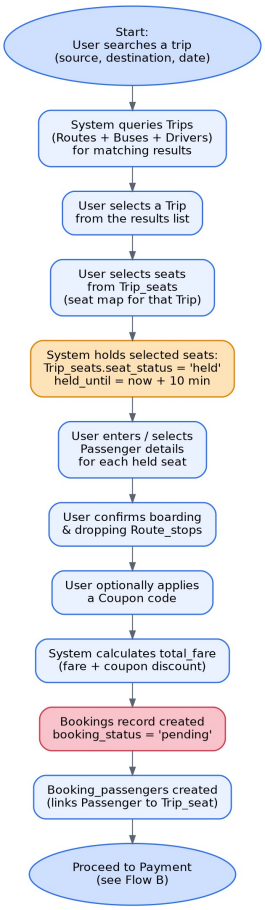


Figure 2 — Search through to a pending Booking with seats held.

Flow B — Payment

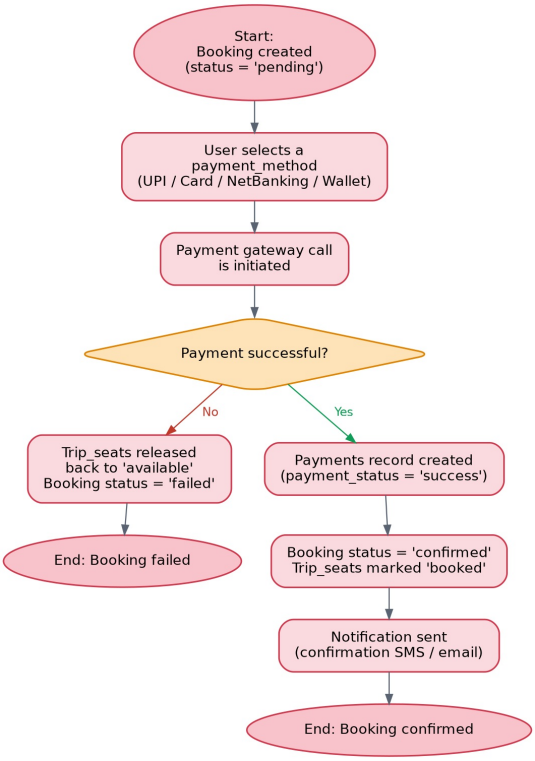


Figure 3 — Payment gateway decision branch: success confirms the Booking and seats; failure releases the held seats.

Flow C — Cancellation

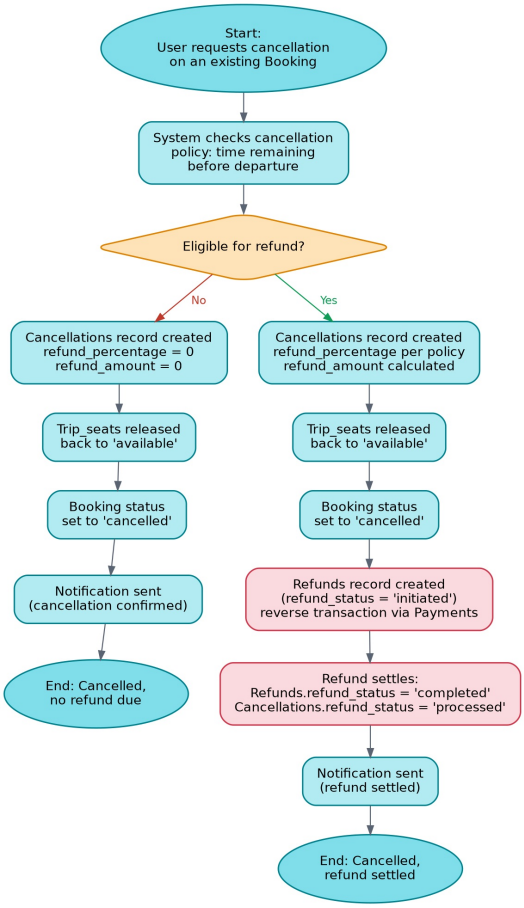


Figure 4 — Cancellation policy check, refund-eligibility branch, seat release, and refund settlement via the Refunds table.

9. Full Column-Level Table Definitions

All 24 tables are shown below as Column, Type, Key, and a plain-English description — no SQL knowledge required to read this section. This includes the 16 core booking-lifecycle tables plus 8 tables benchmarked against real-world platforms (Section 4).

1. Cities

Root of the location hierarchy — every location-based table depends on it.

Column	Type	Key	Description
city_id	SERIAL	PK	Unique identifier for the city
city_name	VARCHAR(100)		Name of the city, e.g. Indore
city_code	VARCHAR(10)	UNIQUE	Short code, e.g. IND — used in route codes
state	VARCHAR(100)		State the city belongs to
country	VARCHAR(100)		Defaults to India
latitude	DECIMAL(9,6)		Map coordinate
longitude	DECIMAL(9,6)		Map coordinate
is_major_hub	BOOLEAN		True if a high-traffic hub city
status	VARCHAR(20)		active / inactive
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

2. Terminals

A physical bus stand located in a city.

Column	Type	Key	Description
terminal_id	SERIAL	PK	Unique identifier for the terminal
city_id	INT	FK → Cities	City this terminal is located in
terminal_name	VARCHAR(150)		e.g. Habibganj ISBT
address	VARCHAR(255)		Full postal address
latitude	DECIMAL(9,6)		Map coordinate
longitude	DECIMAL(9,6)		Map coordinate
status	VARCHAR(20)		active / inactive

3. Bus_types

Controlled reference list for bus categories — added to prevent inconsistent free-text entries.

Column	Type	Key	Description
bus_type_id	SERIAL	PK	Unique identifier for the bus type
type_name	VARCHAR(50)		e.g. AC Sleeper, Non-AC Seater
category	VARCHAR(20)		Seater / Sleeper / Semi-Sleeper
ac_type	VARCHAR(10)		AC / Non-AC
description	VARCHAR(255)		Optional marketing description
status	VARCHAR(20)		active / inactive

4. Buses

A physical bus in the company fleet.

Column	Type	Key	Description
bus_id	SERIAL	PK	Unique identifier for the bus
bus_type_id	INT	FK → Bus_types	Category of the bus
registration_number	VARCHAR(20)	UNIQUE	Government registration plate number
bus_name	VARCHAR(100)		Internal fleet nickname/label
total_seats	INT		Total seat capacity
manufacture_year	INT		Year the bus was manufactured
status	VARCHAR(20)		active / maintenance / retired
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

5. Seat_layout

The seat map of a bus.

Column	Type	Key	Description
seat_id	SERIAL	PK	Unique identifier for the seat
bus_id	INT	FK → Buses	Bus this seat belongs to
seat_number	VARCHAR(10)		Display label, e.g. U1, L2, S14
row_no	INT		Row position in the layout grid
column_no	INT		Column position in the layout grid
deck	VARCHAR(10)		lower / upper (for sleeper buses)
berth_type	VARCHAR(20)		seater / sleeper
seat_position	VARCHAR(10)		window / aisle
is_ladies_seat	BOOLEAN		Reserved for female passengers
price_multiplier	DECIMAL(3,2)		Adjusts fare for premium seats, default 1.00
status	VARCHAR(20)		active / inactive (e.g. seat out of service)

6. Bus_amenities

Feature list for a bus (WiFi, charging point, etc).

Column	Type	Key	Description
amenity_id	SERIAL	PK	Unique identifier for the amenity record
bus_id	INT	FK → Buses	Bus this amenity applies to
amenity_name	VARCHAR(50)		e.g. WiFi, Charging Point, Blanket, CCTV, GPS Tracking
created_at	TIMESTAMP		Record creation time

7. Routes

A path from one city to another.

Column	Type	Key	Description
route_id	SERIAL	PK	Unique identifier for the route
route_code	VARCHAR(20)	UNIQUE	Short code, e.g. BPL-IND-01
route_name	VARCHAR(150)		e.g. Bhopal to Indore via Dewas
source_city_id	INT	FK → Cities	Starting city
destination_city_id	INT	FK → Cities	Ending city
source_terminal_id	INT	FK → Terminals	Default starting terminal
destination_terminal_id	INT	FK → Terminals	Default ending terminal
distance_km	DECIMAL(6,2)		Total route distance
estimated_duration	INTERVAL		Expected journey time
base_fare	DECIMAL(8,2)		Reference fare; can be overridden per trip
route_type	VARCHAR(20)		one-way / round-trip
status	VARCHAR(20)		active / inactive / seasonal / suspended
is_night_route	BOOLEAN		True for overnight sleeper routes
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

8. Route_stops

A stop along a route.

Column	Type	Key	Description
route_stop_id	SERIAL	PK	Unique identifier for the stop record
route_id	INT	FK → Routes	Route this stop belongs to
city_id	INT	FK → Cities	City of this stop
terminal_id	INT	FK → Terminals	Specific terminal, if applicable
stop_sequence	INT		Order of the stop along the route (1, 2, 3...)
stop_type	VARCHAR(20)		boarding / dropping / both
distance_from_origin_km	DECIMAL(6,2)		Cumulative distance — used for segment fares
arrival_offset	INTERVAL		Time since route start
halt_duration_minutes	INT		How long the bus waits at this stop
is_major_stop	BOOLEAN		Whether this stop is searchable/bookable
status	VARCHAR(20)		active / inactive

9. Drivers

A driver/conductor assigned to a trip.

Column	Type	Key	Description
driver_id	SERIAL	PK	Unique identifier for the driver
full_name	VARCHAR(150)		Driver's name
phone_number	VARCHAR(15)	UNIQUE	Contact number, shown to ops for emergencies
license_number	VARCHAR(50)	UNIQUE	Driving license number
license_expiry	DATE		License expiry date, for compliance alerts
experience_years	INT		Years of driving experience
status	VARCHAR(20)		active / on_leave / inactive
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

10. Trips

A scheduled bus run on a date/time — the schema's centerpiece, the bookable unit.

Column	Type	Key	Description
trip_id	SERIAL	PK	Unique identifier for the trip — the bookable unit
route_id	INT	FK → Routes	Route this trip runs on
bus_id	INT	FK → Buses	Bus assigned to this trip
driver_id	INT	FK → Drivers (optional)	Assigned driver, if driver tracking is enabled
departure_date	DATE		Date the trip departs
departure_time	TIME		Scheduled departure time
arrival_date	DATE		Expected arrival date
arrival_time	TIME		Expected arrival time
fare	DECIMAL(8,2)		Fare for this specific trip instance
available_seats	INT		Live counter of open seats, for fast search
status	VARCHAR(20)		scheduled / departed / completed / cancelled
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

11. Trip_seats

Live seat status for a specific trip.

Column	Type	Key	Description
trip_seat_id	SERIAL	PK	Unique identifier for this seat-on-a-trip record
trip_id	INT	FK → Trips	Trip this seat status applies to
seat_id	INT	FK → Seat_layout	Physical seat being tracked
seat_status	VARCHAR(20)		available / held / booked / blocked
held_until	TIMESTAMP		Temporary lock expiry during checkout
price	DECIMAL(8,2)		Final price for this seat on this trip
updated_at	TIMESTAMP		Last update time

12. Users

A customer account.

Column	Type	Key	Description
user_id	SERIAL	PK	Unique identifier for the account
full_name	VARCHAR(150)		Account holder's name
email	VARCHAR(150)	UNIQUE	Login/contact email
phone_number	VARCHAR(15)	UNIQUE	Login/contact phone
password_hash	VARCHAR(255)		Hashed password, never stored in plain text
gender	VARCHAR(10)		Optional profile field
date_of_birth	DATE		Optional profile field
status	VARCHAR(20)		active / blocked
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

13. Passengers

A person traveling — may not be the account holder.

Column	Type	Key	Description
passenger_id	SERIAL	PK	Unique identifier for the passenger
user_id	INT	FK → Users	Account that added this passenger
full_name	VARCHAR(150)		Passenger's name
age	INT		Passenger's age at time of travel
gender	VARCHAR(10)		male / female / other
id_proof_type	VARCHAR(30)		Optional, e.g. Aadhaar, Passport
id_proof_number	VARCHAR(50)		Optional ID document number
created_at	TIMESTAMP		Record creation time

14. Bookings

A confirmed reservation — the core transaction record.

Column	Type	Key	Description
booking_id	SERIAL	PK	Unique identifier for the booking
booking_reference	VARCHAR(20)	UNIQUE	Human-readable reference/PNR shown to the user
user_id	INT	FK → Users	Account that made the booking
trip_id	INT	FK → Trips	Trip being booked
boarding_stop_id	INT	FK → Route_stops	Chosen boarding point
dropping_stop_id	INT	FK → Route_stops	Chosen dropping point
total_seats	INT		Number of seats in this booking
total_fare	DECIMAL(10,2)		Total amount for the booking
booking_status	VARCHAR(20)		pending / confirmed / cancelled / failed
booking_date	TIMESTAMP		When the booking was created
created_at	TIMESTAMP		Record creation time
updated_at	TIMESTAMP		Last update time

15. Booking_passengers

Which passenger sits in which seat, for which booking.

Column	Type	Key	Description
booking_passenger_id	SERIAL	PK	Unique identifier for this seat-passenger link
booking_id	INT	FK → Bookings	Parent booking
passenger_id	INT	FK → Passengers	Passenger occupying the seat
trip_seat_id	INT	FK → Trip_seats	Specific seat reserved
seat_number	VARCHAR(10)		Denormalized label for quick display, e.g. U1
fare	DECIMAL(8,2)		Fare charged for this specific seat
created_at	TIMESTAMP		Record creation time

16. Payments

Money paid for a booking.

Column	Type	Key	Description
payment_id	SERIAL	PK	Unique identifier for the payment
booking_id	INT	FK → Bookings	Booking this payment covers
payment_method	VARCHAR(30)		UPI / Card / NetBanking / Wallet
transaction_id	VARCHAR(100)	UNIQUE	Gateway transaction reference
amount	DECIMAL(10,2)		Amount paid
payment_status	VARCHAR(20)		pending / success / failed / refunded
payment_date	TIMESTAMP		When the payment was processed
gateway_response	TEXT		Raw response for troubleshooting

17. Cancellations

A cancelled booking and its refund info.

Column	Type	Key	Description
cancellation_id	SERIAL	PK	Unique identifier for the cancellation
booking_id	INT	FK → Bookings	Booking being cancelled
cancelled_by	VARCHAR(20)		user / admin / system
cancellation_reason	VARCHAR(255)		Optional reason text
cancellation_date	TIMESTAMP		When the cancellation was requested
refund_percentage	DECIMAL(5,2)		Percentage refunded per policy
refund_amount	DECIMAL(10,2)		Calculated refund amount
refund_status	VARCHAR(20)		pending / processed / failed
refund_date	TIMESTAMP		When the refund was completed

18. Refunds

The actual money-back transaction for a cancellation — tracked separately since a refund can move through multiple processing states.

Column	Type	Key	Description
refund_id	SERIAL	PK	Unique identifier for the refund transaction
cancellation_id	INT	FK → Cancellations	Cancellation this refund settles
payment_id	INT	FK → Payments	Original payment being refunded
refund_amount	DECIMAL(10,2)		Amount actually refunded
refund_method	VARCHAR(30)		original_payment_method / wallet_credit
refund_status	VARCHAR(20)		initiated / processing / completed / failed
refund_transaction_id	VARCHAR(100)	UNIQUE	Gateway/bank reference for the refund
initiated_at	TIMESTAMP		When the refund was triggered
completed_at	TIMESTAMP		When the refund settled

19. Coupons

A discount code and its rules.

Column	Type	Key	Description
coupon_id	SERIAL	PK	Unique identifier for the coupon
coupon_code	VARCHAR(30)	UNIQUE	Code entered at checkout, e.g. FIRST50
description	VARCHAR(255)		Marketing description of the offer
discount_type	VARCHAR(20)		percentage / flat
discount_value	DECIMAL(8,2)		Percentage or flat amount off
max_discount_amount	DECIMAL(8,2)		Cap on discount for percentage coupons
min_booking_amount	DECIMAL(8,2)		Minimum fare required to use the coupon
valid_from	DATE		Coupon activation date
valid_until	DATE		Coupon expiry date
usage_limit_per_user	INT		Max times one user can redeem it
total_usage_limit	INT		Max total redemptions platform-wide
status	VARCHAR(20)		active / inactive / expired
created_at	TIMESTAMP		Record creation time

20. Booking_coupons

Which coupon was used on which booking.

Column	Type	Key	Description
booking_coupon_id	SERIAL	PK	Unique identifier for this coupon application
booking_id	INT	FK → Bookings	Booking the coupon was applied to
coupon_id	INT	FK → Coupons	Coupon that was redeemed
discount_applied	DECIMAL(8,2)		Actual discount amount granted
applied_at	TIMESTAMP		When the coupon was applied

21. Notifications

A sent SMS/email/push message log.

Column	Type	Key	Description
notification_id	SERIAL	PK	Unique identifier for the notification
user_id	INT	FK → Users	Recipient of the notification
booking_id	INT	FK → Bookings	Related booking, if applicable
notification_type	VARCHAR(30)		sms / email / push
title	VARCHAR(150)		Short notification headline
message	TEXT		Full notification body
status	VARCHAR(20)		sent / failed / pending
sent_at	TIMESTAMP		When it was dispatched

22. Reviews_ratings

A passenger's post-trip feedback.

Column	Type	Key	Description
review_id	SERIAL	PK	Unique identifier for the review
booking_id	INT	FK → Bookings	Booking the review is tied to
user_id	INT	FK → Users	Reviewer
trip_id	INT	FK → Trips	Trip being reviewed
rating	INT		1 to 5 star rating
review_text	VARCHAR(500)		Optional written feedback
created_at	TIMESTAMP		Record creation time

23. Admin_users

A staff account for backend operations (standalone).

Column	Type	Key	Description
admin_id	SERIAL	PK	Unique identifier for the staff account
full_name	VARCHAR(150)		Staff member's name
email	VARCHAR(150)	UNIQUE	Login email
password_hash	VARCHAR(255)		Hashed password
role	VARCHAR(30)		super_admin / ops_manager / support_staff
status	VARCHAR(20)		active / inactive
created_at	TIMESTAMP		Record creation time

24. Operators

A bus company — optional, for multi-operator expansion (standalone).

Column	Type	Key	Description
operator_id	SERIAL	PK	Unique identifier for the operator (optional table)
operator_name	VARCHAR(150)		Company name — only needed if going multi-operator
contact_email	VARCHAR(150)		Business contact email
contact_phone	VARCHAR(15)		Business contact number
license_number	VARCHAR(50)		Transport license/permit number
rating	DECIMAL(2,1)		Aggregate operator rating
status	VARCHAR(20)		active / inactive
created_at	TIMESTAMP		Record creation time

10. Assumptions & Constraints

- **Timezone:** All Trips departure/arrival times are stored in a single company-standard timezone (e.g., IST) — no multi-timezone handling, since this is a single-country operation.
- **Currency:** Single currency assumed (INR) — no multi-currency columns on Payments or Routes.
- **Seat locking:** Assumes a temporary "hold" state on Trip_seats during checkout (industry-standard ~5–10 minutes) to prevent double-booking, modeled via the `held_until` timestamp column.
- **Cancellation policy:** Assumed to be a simple time-based refund percentage rule (e.g., more than 24 hours before departure = 90% refund), not a configurable table. Acceptable for MVP; would need a dedicated `cancellation_policy` reference table if rules become more complex.
- **Soft delete:** All master tables (Cities, Terminals, Buses, Routes) use status columns instead of hard deletes, since historical Bookings must retain valid references.
- **Single operator:** No active Operators dependency — the company is the operator, removing multi-tenant complexity throughout the schema. The Operators table remains present but optional for future expansion.